Avaliação exploratória de classificadores Qwen2.5-Coder no PrimeVul: desempenho global, pareado e recursos de implantação

Autoria a confirmar

[1]Programa de Pós-Graduação em Ciência da Computação
Universidade Federal do Maranhão

Abstract. *This study characterizes 3B and 7B Qwen2.5-Coder configurations on PrimeVul, combining global and paired evaluation with deployment resources. Historical records of prompting, a size baseline, three 3B pilots, and three 7B seeds deployed in NF4 and BF16 were consolidated. For the 7B model, mean paired ordering across the three seeds was 50.62% (NF4) and 49.97% (BF16). The size baseline obtained a global AUC of 0.810 and paired ordering of 21.10%. NF4 recorded lower allocated memory but higher amortized time per function. The audit identified five paired correspondences inconsistent with the patches and examined their exclusion as a sensitivity analysis. The findings describe the observed configurations without establishing semantic understanding, configuration equivalence, or a causal quantization effect.*

Resumo. *Este trabalho caracteriza configurações Qwen2.5-Coder de 3B e 7B no PrimeVul, combinando avaliações global e pareada com recursos de implantação. Foram consolidados registros de prompting, uma referência de tamanho, três pilotos 3B e três seeds do 7B em NF4 e BF16. No 7B, as médias de ordenação pareada das três seeds foram 50,62% em NF4 e 49,97% em BF16. A referência de tamanho apresentou AUC global de 0,810 e ordenação pareada de 21,10%. NF4 registrou menor memória alocada, mas maior tempo amortizado por função. A auditoria identificou cinco correspondências pareadas incompatíveis com os patches e examinou sua exclusão como sensibilidade. Os resultados delimitam o comportamento observado, sem demonstrar compreensão semântica, equivalência de configurações ou efeito causal da quantização.*

1. Introdução

A avaliação de modelos de linguagem de código para detecção de vulnerabilidades exige distinguir o desempenho agregado de classificação da capacidade de reconhecer a diferença entre uma função vulnerável e sua versão corrigida. Em conjuntos nos quais as funções não vulneráveis predominam, a acurácia pode permanecer elevada mesmo quando poucas vulnerabilidades são identificadas. Além disso, uma medida agregada como F1 não explicita, isoladamente, o comportamento do detector sob uma restrição de falsos alarmes. Ding et al. (2024) discutem essas limitações e mostram que a composição dos dados e as condições de avaliação alteram a interpretação do desempenho dos modelos. Para essa tarefa, a questão não se restringe a classificar funções corretamente em média, mas envolve examinar quais erros ocorrem e em quais condições o modelo distingue a presença da vulnerabilidade.

O PrimeVul foi desenvolvido para investigar esse problema em funções C/C++ provenientes de projetos de código aberto. O conjunto combina procedimentos de rotulação, remoção de duplicatas e divisão temporal dos dados com diretrizes que complementam as métricas globais. Entre essas diretrizes estão a avaliação de pares rotulados como uma função vulnerável e sua versão corrigida e a análise da taxa de falsos negativos sob uma restrição de falsos positivos (Ding et al., 2024). A avaliação pareada é relevante porque pretende reunir versões de código semelhantes que diferem pela correção da vulnerabilidade e examina as decisões sobre ambas, tornando visíveis erros que uma avaliação agregada não descreve diretamente. A correspondência entre os membros requer conferência na release utilizada, como examinado neste estudo. As avaliações global e pareada oferecem, portanto, informações complementares sobre o comportamento do detector, embora não constituam, por si só, uma demonstração de compreensão semântica do código.

Essa cautela encontra respaldo em estudos sobre as limitações dos detectores aprendidos. Risse e Böhme (2024) observaram, nas técnicas e nos conjuntos avaliados, dificuldades em distinguir funções vulneráveis de suas correções e dependência de características sem relação com a vulnerabilidade, investigada por transformações que preservam a semântica. Huang et al. (2026) estenderam essa investigação a modelos de linguagem ajustados, incluindo Qwen2.5-Coder-7B-Instruct, por meio do TrapEval. Seu protocolo contrasta pares vulnerável/corrigida com exemplos vulneráveis associados a funções não vulneráveis sem correspondência, além de examinar perturbações e supervisão de raciocínio. Esses trabalhos justificam avaliar conjuntamente o desempenho global e o pareado, mas seus resultados não podem ser transferidos automaticamente a outras receitas, variantes de modelo ou condições de implantação.

A análise dessas condições também envolve os recursos computacionais associados à adaptação e à inferência. A família Qwen2.5-Coder disponibiliza modelos especializados em código em diferentes escalas, entre elas as variantes de 3 e 7 bilhões de parâmetros utilizadas neste estudo (Hui et al., 2024). LoRA, ou adaptação de baixo posto, permite ajustar matrizes adicionais mantendo congelados os pesos pré-treinados (Hu et al., 2021). QLoRA combina essa adaptação com uma base quantizada em quatro bits e introduz o formato NormalFloat de quatro bits (NF4) para reduzir a memória necessária ao ajuste (Dettmers et al., 2023). Essas técnicas tornam pertinente examinar qualidade de classificação e recursos nas configurações disponíveis, sem pressupor que a redução de memória preserve o desempenho na detecção de vulnerabilidades ou produza o mesmo comportamento em diferentes formas de implantação.

Nesse contexto, este trabalho tem como objetivo analisar, de forma exploratória, o comportamento das configurações Qwen2.5-Coder de 3B e 7B efetivamente avaliadas no PrimeVul, articulando métricas globais, avaliação pareada e recursos computacionais registrados. A pergunta que orienta o estudo é: como essas configurações se comportam nas avaliações global e pareada, e quais recursos são observados nas formas de implantação executadas? O recorte reúne avaliações por instruções textuais (*prompting*) das variantes Instruct, classificadores ajustados e uma referência de classificação baseada no tamanho das funções em tokens. A inclusão dessa referência permite contextualizar o desempenho dos modelos diante de uma característica simples do código, sem assumir que o tamanho explique todos os erros ou que seu controle elimine outros atalhos.

Os registros experimentais compreendem três receitas exploratórias de ajuste do modelo 3B, cada uma com uma semente de aleatoriedade (*seed*), e uma receita principal do 7B executada com três seeds. Os pilotos 3B incluem amostragem 1:1 entre classes, manutenção da proporção original e amostragem 1:1 com controle de tamanho; o bloco 7B utiliza a última dessas receitas. As diferenças de composição e de orçamento de treinamento restringem a interpretação das comparações entre os pilotos e entre as escalas. Para os modelos 7B, foram registradas avaliações em NF4 com adaptador e em BF16, formato de ponto flutuante de 16 bits, com o adaptador mesclado aos pesos. Como essa comparação modifica tanto a representação dos pesos quanto a incorporação do adaptador, seu objeto são as configurações completas de implantação, sem atribuição causal das diferenças apenas à quantização.

A contribuição do estudo consiste em reunir esse bloco experimental em uma caracterização delimitada, relacionando as perspectivas global e pareada com a referência de tamanho, as repetições do 7B e os registros de recursos. Diferentemente do TrapEval, no qual Huang et al. (2026) construíram os conjuntos V2P e V2N a partir de múltiplas fontes e ajustaram modelos para gerar respostas com ou sem raciocínio supervisionado, este recorte utiliza as avaliações global e pareada do próprio PrimeVul e, no bloco ajustado, uma cabeça de classificação que produz escores. As decisões pareadas de classificação correspondem aos desfechos atribuídos às duas versões; a ordenação, examinada nos classificadores com escores disponíveis, compara os escores da versão vulnerável e da corrigida. As medições de memória e tempo permanecem vinculadas ao ambiente e ao procedimento que as produziram. O estudo não propõe uma nova arquitetura nem um novo teste de compreensão semântica; sua contribuição é empírica e descritiva, sem reivindicação de novidade para a avaliação pareada ou para o uso de adaptação de baixo posto.

O alcance dessa caracterização é limitado a uma família de modelos, a um conjunto de dados e às execuções realizadas. O desenvolvimento teve caráter exploratório, com acesso ao conjunto de teste durante a investigação, o que restringe sua interpretação como avaliação confirmatória independente. As três seeds do 7B permitem examinar as execuções observadas, mas não sustentam uma generalização robusta sobre futuros treinamentos. Da mesma forma, as configurações NF4 e BF16 não abrangem uma comparação completa de métodos de quantização ou uma determinação de implantação ótima. Esses limites orientam a interpretação da evidência e mantêm o objetivo centrado no comportamento verificável dos classificadores e nos recursos registrados.

2. Fundamentação e trabalhos relacionados

A detecção considerada neste estudo é uma tarefa de classificação binária no nível da função: a entrada é código C/C++ e a saída indica a presença ou ausência de vulnerabilidade conforme os rótulos do conjunto. Essa formulação não localiza necessariamente a linha defeituosa, não utiliza todo o contexto do projeto e não oferece uma prova de segurança para funções classificadas como não vulneráveis. No PrimeVul, Ding et al. (2024) associam uma base com rotulação e deduplicação mais cuidadosas a uma avaliação que considera o desbalanceamento entre classes, a restrição de falsos alarmes e os pares de funções rotulados como vulnerável e corrigida. Esses componentes tornam o benchmark pertinente ao estudo, mas precisam ser distinguidos da identidade da release efetivamente utilizada e da qualidade de cada correspondência pareada.

Modelos de linguagem podem realizar essa classificação por geração ou por uma cabeça específica. No primeiro caso, uma instrução solicita uma resposta textual e um parser a converte em rótulo. No segundo, uma camada de classificação produz logits para as classes, permitindo obter um escore contínuo e calibrar limiares. As variantes Base e Instruct da família Qwen2.5-Coder, descrita por Hui et al. (2024), viabilizam esses dois usos. A especialização do modelo em código não constitui evidência suficiente de capacidade para identificar vulnerabilidades, que permanece uma propriedade a avaliar no protocolo definido.

LoRA representa uma atualização de pesos por matrizes de baixo posto. Para uma matriz pré-treinada W_0 de dimensões $d \times k$, a atualização é representada por $\Delta W = BA$, de modo que o peso adaptado é W_0 + BA, com posto r muito menor que o mínimo entre d e k; os pesos originais permanecem congelados e os parâmetros adicionais são ajustados à tarefa (Hu et al., 2021). QLoRA utiliza uma base congelada quantizada e adaptadores treináveis, distinguindo o formato de armazenamento do formato de computação. Dettmers et al. (2023) apresentam NF4 e a quantização dos próprios fatores de quantização para reduzir a memória associada aos pesos. Neste estudo, a adaptação empregou NF4 e computação BF16, mas o otimizador foi AdamW convencional: não se atribui à implementação local a adoção de todos os componentes do trabalho original, como otimizadores paginados.

O posicionamento científico resulta de diferenças concretas de desenho. PrimeVul fornece dados e diretrizes de avaliação; Risse e Böhme (2024) investigam limitações de detectores aprendidos e a influência de características sem relação com a vulnerabilidade; Huang et al. (2026) examinam modelos ajustados no TrapEval, com conjuntos V2P/V2N de múltiplas fontes, perturbações e supervisão de raciocínio. O presente artigo caracteriza um bloco de execuções Qwen no teste do próprio PrimeVul, combinando scores classificatórios, seeds observadas e recursos registrados. Não executa o protocolo TrapEval, não compara diretamente números obtidos em diferentes benchmarks e não interpreta ordenação próxima de 50% como diagnóstico de Semantic Trap. O contraste com esses estudos orienta a cautela necessária à leitura dos resultados, sem sustentar uma reivindicação de pioneirismo.

3. Método

3.1. Delineamento, dados e rastreabilidade

Realizou-se uma consolidação retrospectiva dos experimentos existentes, com acesso prévio ao teste durante a investigação. O estudo, portanto, tem caráter exploratório. Foram incluídas quatro avaliações por prompting (3B e 7B Instruct, cada um no teste global e no pareado), três pilotos 3B e seis avaliações de três adaptadores 7B, cada um implantado em NF4 e BF16, além da referência de tamanho. Para os classificadores e a referência de tamanho, cada avaliação reúne seus respectivos splits. Linhas de depuração, a release v0.1 e execuções descartadas ficaram fora da análise principal. Nesta consolidação não foram realizados treinamento, inferência, recuperação de pesos ou novas medições em GPU; os cálculos utilizaram exclusivamente os registros disponíveis e processamento em CPU.

A release original foi recuperada da pasta pública indicada pelo repositório oficial do PrimeVul. O treino contém 184.427 funções, sendo 5.574 vulneráveis; a validação

global contém 25.430, sendo 699 vulneráveis; o teste global contém 25.911, sendo 695 vulneráveis, aproximadamente 2,68%. Os arquivos pareados têm 4.354 pares de treino, 562 de validação e 564 de teste. Os hashes SHA-256 completos calculados nesta etapa coincidiram com os prefixos históricos de 12 caracteres disponíveis para treino, teste global e teste pareado. Essa correspondência reforça a rastreabilidade da recuperação; não transforma prefixos históricos em hashes completos que teriam sido gravados na execução.

Cada log foi alinhado à fonte por posição original, identificador idx, projeto, commit e rótulo. A ordem de processamento do prompting, registrada em i, não foi utilizada como identidade. Conferiram-se cobertura integral, unicidade de posições e ausência de divergências desses campos. O campo func_hash nulo não foi empregado como prova. Os pares foram reconstruídos pela ordem da release oficial, exigindo duas posições consecutivas, classes opostas e projeto/commit coincidentes, sem fallback por agrupamento arbitrário. O arquivo de teste reutiliza duas identidades de funções corrigidas em pares distintos; a validação pareada reutiliza uma identidade em cinco pares. Essas ocorrências correspondem ao mesmo código e foram documentadas, preservando a unidade posicional e os denominadores oficiais.

A auditoria distinguiu identidade documental e correspondência de correção. Quinze pares de teste apresentam diferenças de cabeçalho ou assinatura. A inspeção focalizada dos diffs e, nos cinco casos suspeitos, dos arquivos e commits originais, identificou que a função rotulada como corrigida já existia antes do patch e não era a versão corrigida da função vulnerável: libjpeg (2022), PHP (2011), rizin (2022), Linux (2008) e drogon (2022). Os registros e as fontes primárias dessas decisões integram o suplemento. Mantiveram-se os 564 pares como referência do conjunto oficial e calculou-se uma sensibilidade que remove somente esses cinco pares, com 559 restantes. Não foram substituídas funções por contrapartes sem previsões, nem se certificou por essa inspeção a correção semântica de todos os pares restantes.

3.2. Configurações e ajuste

As avaliações por prompting utilizaram Qwen2.5-Coder-3B-Instruct e 7B-Instruct em FP16. A instrução solicitou uma única palavra, YES ou NO, para cada função, sem exemplos; a geração foi determinística, com até cinco novos tokens. O código foi limitado a 8.192 tokens nas rodadas incluídas, conforme os registros históricos. O parser aceita a primeira palavra reconhecida e, quando necessário, termos de decisão nos primeiros 60 caracteres; respostas indefinidas recebem rótulo não vulnerável e são contabilizadas. Os quatro logs incluídos não registraram respostas indefinidas. O texto integral da instrução e o parser permanecem no código histórico e no suplemento, permitindo avaliar essa política.

Para os classificadores, utilizaram-se os modelos Base Qwen2.5-Coder-3B e 7B com uma cabeça de duas saídas. O escore de vulnerabilidade foi definido pela diferença entre o logit vulnerável e o não vulnerável. O ajuste empregou base NF4, dupla quantização, computação BF16 e cabeça em precisão plena. Aplicou-se LoRA às projeções de atenção q, k, v e o e às projeções gate, up e down, com r = 16, α = 32 e dropout de 0,05. A cabeça classificatória também foi treinada. O otimizador AdamW utilizou taxa inicial de 0,0002, weight decay zero, aquecimento de 3% dos passos e decaimento linear;

o lote efetivo continha 16 funções, com divisão por orçamento de tokens, checkpointing de gradientes e corte da norma do gradiente em 1,0. A perda foi entropia cruzada, sem aumento do peso da classe vulnerável. Os demais argumentos, revisões e versões estão preservados nas receitas.

Tabela 1. Configurações de ajuste. Tempo corresponde ao treino registrado; custo inclui o tempo total registrado da rodada, ao preço histórico de US$ 0,50/h. As horas não representam orçamento igual entre receitas.

Configuração	Vulneráveis	Benignas	Épocas	Selecionada	Treino (h)	Custo total (US$)
3B 1:1	5574	5574	3	1	3,37	1,85
3B real	5574	178853	1	1	10,16	5,14
3B 1:1 tamanho	5574	5574	3	3	4,98	2,65
7B 1:1 tamanho s1	5574	5574	3	3	7,89	4,21
7B 1:1 tamanho s2	5574	5574	3	2	7,89	4,21
7B 1:1 tamanho s3	5574	5574	3	3	7,91	4,23

Todos os pilotos 3B utilizaram seed 1. A receita 1:1 sorteada reteve todas as funções vulneráveis e sorteou igual quantidade de funções não vulneráveis; a receita de proporção real utilizou todo o treino; a receita 1:1 com controle de tamanho escolheu, sem reutilização, funções não vulneráveis com comprimento tokenizado mais próximo ao de cada vulnerável. O comprimento considerado já estava limitado a 2.048 tokens. Esse controle procura reduzir diferenças de tamanho entre classes no treino, sem remover necessariamente outras características correlacionadas com os rótulos. O modelo 7B utilizou essa última receita nas seeds 1, 2 e 3. Como composição, número de exemplos, duração e repetições diferem entre receitas, essas comparações não isolam o efeito causal da escala ou da amostragem.

A seleção histórica de época utilizou ordenação estrita no pareado da validação e AUC como desempate. A checagem global por época empregou 699 vulneráveis e 2.796 não vulneráveis; essa amostra não é a validação global completa usada na calibração posterior. As épocas selecionadas estão na Tabela 1. A correção retrospectiva de 06/10 atribuiu meio crédito aos empates. Nos quatro treinos com controle de tamanho, os registros de empates por época permitiram reconstruir esse ajuste e a época selecionada permaneceu a mesma. Nos pilotos sem esse registro não foi possível conferir a seleção corrigida; mantiveram-se os checkpoints históricos, sem afirmar que a revisão seria inócua em todas as receitas.

Para cada checkpoint 7B selecionado, a avaliação NF4 manteve a base quantizada e o adaptador separado. A avaliação BF16 carregou a base em BF16 e mesclou o adaptador aos pesos. A comparação é feita dentro da mesma seed e época, mas envolve tanto a representação da base quanto a forma de incorporar o adaptador. Não se trata de um experimento que altera exclusivamente o número de bits.

3.3. Calibração e métricas

Os classificadores e a referência de tamanho utilizaram contexto de 2.048 tokens, preservando o início da função. A referência atribui como escore o número de tokens visíveis após esse corte. A revisão histórica do tokenizer não foi registrada separadamente para o

baseline; recuperou-se o artefato da revisão do modelo 3B registrada nos classificadores e verificou-se coincidência de todos os comprimentos do teste e do pareado com os logs. Os arquivos tokenizer.json das revisões recuperadas de 3B e 7B são idênticos. A validação do baseline foi recontada em CPU, e seus valores de calibração e teste reproduziram os números publicados.

A regra classificatória é $s > t$. Foram mantidas duas calibrações distintas, ambas na validação global completa. Para F1, ordenaram-se os scores e escolheram-se cortes entre valores distintos, incluindo o grupo empatado acima do corte; em empate entre valores máximos de F1, a implementação escolhe o primeiro corte encontrado na ordem decrescente. Numericamente, o limiar fica no valor representável imediatamente inferior ao último score incluído. Para limitar falsos positivos no dev, o limiar foi o score da função não vulnerável que delimita o número máximo de alarmes, dado por piso de 0,005 vezes a quantidade dessa classe. A regra estrita exclui os scores empatados nesse limiar. Em ambos os casos, o limiar escolhido no dev foi aplicado sem recalibração ao teste global e ao pareado.

No teste global, calcularam-se precisão, revocação, F1, taxa de falsos positivos (FPR) e área sob a curva ROC (AUC), quando havia scores. Também se calculou a taxa de falsos negativos (FNR) no limiar calibrado para FPR $\leq 0{,}5\%$ no dev, acompanhada da FPR realmente observada no teste. Esse ponto não deve ser descrito como cumprimento da restrição no teste quando sua FPR ultrapassa 0,5%. O script oficial de VD-Score escolhe um ponto da curva ROC no conjunto fornecido e aplica uma regra inclusiva; o presente procedimento próprio separa calibração e teste e aplica $s > t$. Portanto, seus números não são tratados como reprodução direta do VD-Score oficial. Valores históricos de oráculo, com limiar escolhido no teste, não foram usados como resultados principais.

No pareado, orientando primeiro a função rotulada como vulnerável, as decisões (1,0), (1,1), (0,0) e (0,1) definem P-C, P-V, P-B e P-R. As tabelas principais dos classificadores utilizam o limiar F1 do dev; as categorias históricas no limiar zero foram reconstruídas e preservadas no suplemento. Para prompting, permanece a decisão do parser. Essas categorias não devem ser confundidas com a ordenação contínua, definida por $A_{\text{par}} = [\sum_i \mathbf{1}(s_{v,i} > s_{c,i}) + 0{,}5 \sum_i \mathbf{1}(s_{v,i} = s_{c,i})]/N$. A ordenação não depende do limiar e atribui meio crédito a empates nos scores disponíveis. Seu valor de referência 50% pressupõe simetria; não corresponde a uma prova ou a uma medida direta de compreensão semântica. Uma referência P-C de 25% exigiria decisões independentes e equiprováveis e não foi adotada como limiar universal.

Os scores históricos foram gravados com cinco casas decimais. A reconstrução utilizou essa precisão disponível, sem criar casas internas ausentes ou ajustar epsilon ao resultado desejado. Os limiares derivados foram armazenados sem novo arredondamento. Contagens globais históricas, categorias pareadas no limiar original, AUC, F1 calibrado e FNR/FPR da calibração reproduziram os registros na precisão publicada. Essa coincidência não garante a recuperação da precisão interna de todas as operações; as novas categorias calibradas devem ser entendidas como resultados derivados dos logs disponíveis.

3.4. Incerteza, sensibilidades e recursos

A variação entre seeds foi descrita por valores individuais, média e desvio padrão amostral das três execuções 7B, sem agregar pilotos 3B como repetições. A incerteza amostral da ordenação foi examinada por bootstrap percentil de 95%, com 10.000 réplicas e seed 20261008. Reamostraram-se os 550 grupos projeto/commit dos 564 pares, mantendo todas as funções de cada grupo e os mesmos índices entre variantes e seeds. Cada réplica estima a média ponderada pelos pares efetivamente reamostrados, preservando o vetor de resultados das três seeds. Assim, a média não representa um ensemble nem uma amostra triplicada. Calcularam-se intervalos por modelo, da média das seeds e das diferenças NF4 - BF16 dentro da seed e para a média. A análise é condicional aos modelos treinados e aos limiares observados, sob a hipótese de grupos de commit independentes/intercambiáveis; dependência entre commits do mesmo projeto continua possível. Uma reamostragem por par foi preservada como sensibilidade. A comparação dos endpoints entre 5.000 e 10.000 réplicas variou no máximo 0,10 ponto percentual por modelo, arredondada a duas casas. Não foram transportados p-valores das notas históricas nem realizados testes adicionais de hipótese.

Para examinar truncamento, repetiu-se a ordenação nos 465 pares em que ambas as versões cabem em 2.048 tokens, mantendo as mesmas regras. Foram comparadas as sequências tokenizadas dos membros para distinguir igualdade de entrada e igualdade de score. A sensibilidade de correspondência removeu os cinco pares documentados; seu cruzamento com o recorte sem truncamento contém 460 pares. Esses recortes retrospectivos mudam a composição do teste e não certificam os pares restantes ou a generalização para funções longas.

Os recursos provêm das execuções históricas em NVIDIA A40, com PyTorch 2.8.0+cu128, CUDA 12.8, Transformers 5.17.0, tokenizers 0.23.2, PEFT 0.21.0 e bitsandbytes 0.50.2. Nos classificadores, a memória após carga é a memória CUDA alocada, em GB decimais. O pico foi reiniciado após carga/merge e observado na janela que abrange validação, teste e pareado; não inclui necessariamente o pico transitório do merge. O tempo amortizado do teste corresponde ao tempo de pontuação dividido pelo número de funções, em lotes limitados a 32.768 tokens, incluindo preenchimento. O custo computacional estimado usa tempo de pontuação do teste e preço histórico de US$ 0,50/h. Não representa memória total/reservada, latência individual, energia ou fatura real. Os tempos de prompting, obtidos por geração sequencial e em outro contexto, não foram comparados diretamente aos dos classificadores.

4. Resultados

4.1. Conferência e avaliação global

A junção dos 33 arquivos de previsão das 14 avaliações selecionadas com os splits oficiais não apresentou divergências de posição, idx, projeto, commit ou rótulo e manteve cobertura integral. A reconstrução não exigiu alterar CSVs ou registros históricos. O teste pareado tem 1.128 posições, mas 1.126 identidades idx distintas por causa das duas reutilizações oficiais documentadas. A distinção entre correspondência documental e correspondência de correção é relevante: os cinco casos confirmados pelos patches afetam a interpretação do pareado, embora não indiquem erro de alinhamento dos logs.

Tabela 2. Prompting no teste global (25.911 funções) e categorias no teste pareado (564 pares), pela regra YES/NO. Contexto do código: 8.192 tokens.

Modelo	F1	Precisão	Revocação	FPR (%)	P-C	P-V	P-B	P-R
Prompt 3B	0,119	0,102	0,144	3,49	16	67	463	18
Prompt 7B	0,062	0,038	0,160	11,00	14	78	454	18

No prompting global, o 3B apresentou F1 de 0,119 e o 7B de 0,062. A maior revocação do 7B, 0,160 frente a 0,144, foi acompanhada de FPR de 11,00%, contra 3,49% no 3B, reduzindo a precisão. Nos pares, a categoria P-B concentrou 463 e 454 dos 564 casos, respectivamente; apenas 16 e 14 tiveram decisões P-C. Esses resultados descrevem a instrução e o parser utilizados, e não o desempenho de toda forma de prompting possível.

Tabela 3. Teste global, 25.911 funções. F1/precisão/revocação/FPR usam o limiar de maior F1 no dev; FNR* e FPR* usam outro limiar, calibrado para FPR $\leq$ 0,5% no dev. FPR* é o valor efetivamente observado no teste. AUC usa os scores disponíveis.

Configuração	F1	Precisão	Revocação	FPR (%)	AUC	FNR* (%)	FPR* (%)
Tamanho	0,200	0,152	0,294	4,53	0,810	100,00	0,00
3B 1:1	0,195	0,142	0,312	5,19	0,795	94,10	0,54
3B real	0,262	0,200	0,380	4,19	0,851	88,06	0,65
3B 1:1 tamanho	0,121	0,083	0,224	6,86	0,746	96,26	0,63
7B NF4 s1	0,114	0,077	0,222	7,36	0,738	97,99	0,69
7B BF16 s1	0,119	0,081	0,229	7,19	0,740	98,13	0,67
7B NF4 s2	0,142	0,103	0,229	5,47	0,726	98,71	0,38
7B BF16 s2	0,148	0,115	0,207	4,41	0,728	98,71	0,38
7B NF4 s3	0,127	0,084	0,258	7,75	0,765	98,27	0,56
7B BF16 s3	0,124	0,087	0,220	6,39	0,765	98,27	0,52

A referência de tamanho alcançou F1 de 0,200 e AUC de 0,810. Entre os pilotos 3B, a receita de proporção real apresentou o maior F1 global observado, 0,262, e AUC de 0,851; a receita 1:1 com controle de tamanho apresentou F1 de 0,121 e AUC de 0,746. Esse contraste é descritivo, pois número de exemplos, duração, composição e repetições diferem. Nas seis avaliações 7B, F1 variou entre 0,114 e 0,148 e AUC entre 0,726 e 0,765. Não houve predominância global consistente de NF4 ou BF16 entre as três seeds.

O limiar F1 e o limiar de restrição de falsos positivos expressam escolhas diferentes. Na receita 3B real, por exemplo, FNR foi 88,06% no limiar de FPR calibrado no dev, acompanhado de FPR de 0,65% no teste, acima da restrição nominal. No 7B, FNR variou de 97,99% a 98,71%; somente as avaliações da seed 2 tiveram FPR de teste inferior a 0,5%. A referência de tamanho resultou em FNR de 100% e FPR zero nesse ponto: seu limiar foi o teto de 2.048 tokens e, pela regra estrita, nenhum escore truncado o ultrapassou. Esse efeito não demonstra que comprimento seja incapaz de discriminar classes; sua AUC e seu F1 sob outra calibração mostram que as interpretações dependem da regra adotada.

4.2. Avaliação pareada e sensibilidades

Tabela 4. Teste pareado oficial, 564 pares. Categorias P-* usam o limiar F1 do dev; ordenação usa meio crédito para empates e independe de limiar. A coluna 465 considera apenas pares com ambas as versões não truncadas.

Configuração	P-C	P-V	P-B	P-R	Empates	Ord. 564 (%)	Ord. 465 (%)
Tamanho	2	156	398	8	124	21,10	15,16
3B 1:1	13	155	372	24	52	42,38	39,03
3B real	6	200	344	14	43	37,50	34,09
3B 1:1 tamanho	6	120	422	16	37	49,20	48,71
7B NF4 s1	7	116	435	6	39	55,23	54,09
7B BF16 s1	6	125	425	8	39	53,28	51,94
7B NF4 s2	5	128	423	8	40	45,04	42,37
7B BF16 s2	7	113	434	10	34	44,68	41,94
7B NF4 s3	6	134	412	12	32	51,60	51,61
7B BF16 s3	8	113	434	9	32	51,95	52,04

A ordenação da referência de tamanho foi 21,10%, embora sua AUC global tenha sido 0,810. O piloto 3B real, que apresentou o maior F1 global observado, ordenou 37,50% dos pares com meio crédito para empates. O piloto 3B com controle de tamanho atingiu 49,20%. No 7B, a ordenação variou de 45,04% a 55,23% em NF4 e de 44,68% a 53,28% em BF16. As decisões P-C no limiar F1 do dev permaneceram baixas: de cinco a oito pares nas seis avaliações. Uma ordem relativa favorável dos scores não implica que o limiar escolhido produza simultaneamente as duas decisões corretas.

Tabela 5. Ordenação pareada do 7B: média das três seeds, dispersão descritiva entre seeds e IC percentil de 95% por bootstrap de 550 grupos projeto/commit (10.000 réplicas), condicional aos três modelos observados. O intervalo da diferença mantém alinhamento entre variantes.

Medida	Estimativa (%) ou p.p.	DP seeds (p.p.)	IC95% inferior	IC95% superior
NF4	50,62	5,17	47,92	53,33
BF16	49,97	4,63	47,14	52,73
NF4 - BF16 (p.p.)	0,65	—	-1,22	2,54

A média de ordenação das três seeds foi 50,62% para NF4, com desvio padrão de 5,17 pontos percentuais, e 49,97% para BF16, com desvio padrão de 4,63 pontos. Os intervalos condicionais por grupo de commit são apresentados na Tabela 5. A diferença média NF4 - BF16 foi 0,65 ponto percentual, com intervalo de -1,22 a 2,54 pontos. Esse intervalo abrange diferenças em ambas as direções e não estabelece equivalência. A dispersão entre as seeds observadas é distinta dessa incerteza condicional do teste e não foi substituída pelo intervalo da média.

Nos 465 pares sem truncamento, a ordenação NF4 foi 54,09%, 42,37% e 51,61%, nas seeds 1, 2 e 3; BF16 apresentou 51,94%, 41,94% e 52,04%. A retirada das funções

truncadas reduziu principalmente os resultados da seed 2. A inspeção das entradas encontrou 32 pares com sequências idênticas após o corte, enquanto as seis avaliações 7B registraram de 32 a 40 empates de score. Portanto, parte dos empates corresponde a entradas idênticas, mas scores empatados não são, isoladamente, prova dessa identidade. A mudança de denominador também impede atribuir toda diferença da sensibilidade exclusivamente ao truncamento.

Ao remover os cinco pares cuja correspondência não representa o patch da mesma função, a ordenação ficou entre 45,26% e 55,19% em NF4 e entre 44,90% e 53,22% em BF16. A maior alteração absoluta nos seis resultados 7B foi 0,22 ponto percentual, arredondada a duas casas. O padrão descritivo de variação entre seeds permaneceu, mas esse resultado não valida os outros 559 pares nem recupera avaliações das verdadeiras versões corrigidas. A tabela completa dos recortes 564/559/460 integra o suplemento.

4.3. Recursos de implantação

Tabela 6. Recursos históricos do 7B na NVIDIA A40, Transformers, 32.768 tokens por lote e contexto de 2.048. GB decimais de memória alocada após carga e pico na janela que abrange dev/teste/pareado. Tempo amortizado do teste em lotes; custo computacional estimado ao preço histórico US$ 0,50/h. As seeds não são repetições independentes de desempenho.

Configuração	Após carga (GB)	Pico (GB)	Tempo/função (ms)	Custo teste (US$)
7B NF4 s1	4,63	12,49	59,6	0,2145
7B BF16 s1	14,15	20,76	46,8	0,1684
7B NF4 s2	4,63	12,49	59,8	0,2152
7B BF16 s2	14,15	20,76	46,9	0,1687
7B NF4 s3	4,63	12,49	60,0	0,2160
7B BF16 s3	14,15	20,76	47,1	0,1696

NF4 registrou 4,63 GB alocados após carga e pico de 12,49 GB, contra 14,15 GB e 20,76 GB em BF16 mesclada. A redução observada após carga foi aproximadamente 67%, e a diferença dos picos correspondeu a cerca de 40%. Entretanto, o tempo amortizado de pontuação por função foi maior em NF4: 59,6–60,0 ms, frente a 46,8–47,1 ms em BF16. No mesmo preço histórico por hora, o custo computacional estimado de pontuação do teste acompanhou essa diferença de tempo.

Os registros mostram, assim, uma troca entre memória alocada e tempo nessas formas de implantação. Como não houve repetições independentes de desempenho, controle completo das condições do hardware ou isolamento entre representação dos pesos e merge, as pequenas diferenças entre seeds não foram tratadas como um benchmark de latência. A evidência sustenta a descrição do ambiente executado, sem determinar a configuração de melhor custo para outros ambientes ou cargas.

5. Discussão

O conjunto dos resultados mostra que desempenho global e comportamento pareado oferecem leituras diferentes. A referência de tamanho discrimina as classes globalmente, mas ordena desfavoravelmente os pares oficiais; o piloto 3B real apresenta o maior F1

global observado e ordenação de 37,50%; os classificadores com controle de tamanho se aproximam de 50% no pareado, sem melhor F1 global. Esses contrastes são compatíveis com a necessidade de avaliações complementares defendida por Ding et al. (2024). Não demonstram, contudo, que um modelo específico tenha aprendido somente comprimento, que o controle de tamanho tenha removido todos os atalhos ou que as diferenças resultem exclusivamente da receita de treino.

A calibração também altera o objeto observado. No limiar histórico zero, as 564 decisões pareadas do piloto 3B real foram P-B. No teste global, seu F1 foi 0,262 no limiar reconstruído da validação. Nos modelos com controle de tamanho, a adoção do limiar F1 reduz a frequência de decisões positivas e modifica as categorias pareadas, sem alterar a ordenação dos scores. Relatar apenas F1 no limiar zero ou apenas ordenação excluiria parte relevante dessa relação. Sob a política de poucos falsos alarmes calibrada no dev, as altas taxas de falsos negativos e a extrapolação da FPR no teste explicitam outra limitação das configurações examinadas.

A comparação NF4/BF16 mostra que menor memória observada não implica menor tempo nem preservação garantida de qualidade. A diferença média de ordenação foi pequena em relação à dispersão entre as seeds, mas o intervalo não confirma equivalência. A comparação por checkpoint evita confundir treinos diferentes, sem isolar a quantização da mudança de construção do modelo. Portanto, o artigo caracteriza duas configurações disponíveis, e não uma curva de Pareto ou uma conclusão geral sobre métodos de quantização.

O pareado também depende da composição do benchmark. Os cinco casos confirmados nos commits originais mostram que a identidade correta da release e o fechamento das contagens não bastam para garantir a relação vulnerável/correção em cada par. A sensibilidade documentada alterou pouco a ordenação do bloco 7B, mas não elimina o problema conceitual. A interpretação deve permanecer ligada ao conjunto oficial e aos recortes explicitados. Assim como Risse e Böhme (2024) e Huang et al. (2026) investigam limites dos detectores em desenhos específicos, este estudo evidencia limites de avaliação das configurações executadas; a comparação não autoriza transferir seus diagnósticos semânticos nem seus resultados quantitativos ao bloco local.

6. Ameaças à validade

O acesso ao teste durante a investigação, a escolha exploratória de receitas e a reconstrução retrospectiva das políticas restringem a validade confirmatória. A utilização do dev na escolha de limiares evita recalibrar esses pontos no teste nesta etapa, mas não apaga o contato anterior com seus resultados. Os checkpoints históricos foram preservados; a correção de empates foi explicitada sem atribuir retrospectivamente a todas as decisões o critério posterior.

Há uma seed por piloto 3B e três no bloco 7B. As receitas diferem em composição e orçamento; as variantes Base/Instruct e os contextos de 2.048/8.192 tokens também diferem. Consequentemente, as comparações não isolam escala, estratégia de ajuste ou prompting. O bootstrap condiciona-se às execuções disponíveis e não estima com robustez a distribuição de futuros treinamentos. A reamostragem por commit acomoda a dependência dentro desses grupos e as reutilizações identificadas, mas não elimina relações entre commits de um mesmo projeto. Intervalos próximos da referência simétrica

não constituem teste de equivalência ou evidência de ausência de capacidade semântica.

A identidade dos logs foi conferida integralmente, mas não foi realizada uma nova auditoria humana de todos os rótulos. A inspeção de correspondência foi focalizada nos cabeçalhos divergentes e produziu uma sensibilidade retrospectiva. Não representa uma avaliação sistemática da taxa de erro de rotulação do PrimeVul. O truncamento remove contexto, e o recorte de funções curtas modifica a amostra. A divisão temporal e a deduplicação do dataset não demonstram ausência de contaminação do pré-treinamento dos modelos.

A rastreabilidade histórica registra prefixos de revisões e hashes, sem uma revisão de código identificada por execução ou uma revisão própria de tokenizer em todas as rodadas. Os scores arredondados limitam a recuperação numérica, ainda que as métricas principais coincidam na precisão publicada. Recursos foram registrados em uma GPU e um motor, sem repetições independentes de desempenho; o pico não abrange necessariamente o merge e o tempo amortizado não é latência individual. Não foram medidos energia, tarifas atuais, faturamento real ou outros métodos de quantização. Esses limites restringem a validade externa ao bloco e às condições descritas.

7. Conclusão

Este trabalho consolidou as execuções Qwen2.5-Coder disponíveis no PrimeVul e caracterizou desempenho global, comportamento pareado e recursos de implantação com políticas de decisão explícitas. A reconstrução reproduziu as métricas principais na precisão publicada e separou categorias históricas de resultados calibrados na validação. O maior F1 global observado, 0,262 no piloto 3B de proporção real, coexistiu com ordenação pareada de 37,50%, enquanto as médias do bloco 7B foram 50,62% em NF4 e 49,97% em BF16, com variação entre as três seeds. Esses contrastes sustentam a utilização conjunta das duas perspectivas, sem atribuição causal às receitas ou à escala.

Nas implantações 7B registradas, NF4 utilizou menos memória alocada, mas apresentou maior tempo amortizado de pontuação por função que BF16 mesclada. A diferença de ordenação não estabeleceu equivalência entre as configurações. A auditoria também identificou reutilizações de membros e cinco correspondências incompatíveis com os patches da mesma função no conjunto pareado oficial; sua exclusão foi apresentada como sensibilidade, preservando os registros originais e limitando as conclusões.

A contribuição é uma caracterização empírica delimitada e rastreável do bloco executado. Sua continuidade requer avaliação confirmatória com escolhas anteriores ao acesso ao teste, verificação mais ampla das correspondências pareadas e medições repetidas sob condições controladas. Outros modelos, quantizações e medidas de energia pertencem a essa ampliação futura e não foram apresentados como resultados deste artigo.

Referências

Dettmers, T., Pagnoni, A., Holtzman, A. e Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314v1. https://arxiv.org/abs/2305.14314v1.

Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B. e Chen, Y. (2024). Vulnerability Detection with Code Language Models: How Far Are We? arXiv:2403.18624v2. https://arxiv.org/abs/2403.18624v2.

drogon (2022). Prevent malformed upload path causing arbitrary write (#1174). Commit 3c785326c63a34aa1799a639ae185bc9453cb447. Fonte primária: https://github.com/drogonframework/drogon/commit/3c785326c63a34aa1799a639ae185bc9453cb447.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. e Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685v2. https://arxiv.org/abs/2106.09685v2.

Huang, F., Sun, Y., Zhang, F., Yang, Z., Liu, H. e Liu, Y. (2026). Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap. arXiv:2601.22655v3. https://arxiv.org/abs/2601.22655v3.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J. e Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186v3. https://arxiv.org/abs/2409.12186v3.

libjpeg (2022). The code now checks for consistency of the MCU sizes across. Commit 187035b9726710b4fe11d565c7808975c930895d. Fonte primária: https://github.com/thorfdbg/libjpeg/commit/187035b9726710b4fe11d565c7808975c930895d.

Linux (2008). sctp: Fix kernel panic while process protocol violation parameter. Commit ba0166708ef4da7eeb61dd92bbba4d5a749d6561. Fonte primária: https://github.com/torvalds/linux/commit/ba0166708ef4da7eeb61dd92bbba4d5a749d6561.

PHP (2011). Fix segfault in older versions of OpenSSL (before 0.9.8i). Commit 095cbc48a8f0090f3b0abc6155f2b61943c9eafb. Fonte primária: https://github.com/php/php-src/commit/095cbc48a8f0090f3b0abc6155f2b61943c9eafb.

Risse, N. e Böhme, M. (2024). Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection. 33rd USENIX Security Symposium, pp. 4247–4264. https://www.usenix.org/conference/usenixsecurity24/presentation/risse.

rizin (2022). fix #2964 - double-free in bin_qnx.c. Commit 58926dffbe819fe9ebf5062f7130e026351cae01. Fonte primária: https://github.com/rizinorg/rizin/commit/58926dffbe819fe9ebf5062f7130e026351cae01.